

# CODEFEST AD ASTRA 2026 — Etapa 1

Documento Técnico: Construcción de la Base de Conocimiento

Equipo Laka

## 1. Resumen del sistema

El sistema construye una base de conocimiento vectorial sobre el corpus completo provisto por ADL (1.826 documentos declarados; 1.780 efectivamente indexados tras exclusiones por idioma) y expone un módulo de recuperación que responde las 50 consultas de evaluación en el formato exigido por la Sección 9. El pipeline completo consta de cuatro etapas: (1) extracción y limpieza por formato, (2) chunking semántico con límite de 512 tokens, (3) codificación con un encoder multilingüe e indexación en FAISS, y (4) recuperación y agregación a nivel de documento. Cada etapa fue validada de forma independiente antes de pasar a la siguiente, y dos hipótesis de mejora sobre la etapa de agregación fueron probadas empíricamente contra las 50 consultas reales y descartadas por evidencia de que empeoraban el resultado agregado (Sección 7).

|                        |                                                              |
|------------------------|--------------------------------------------------------------|
| Chunks indexados       | 330.416                                                      |
| Documentos únicos      | 1.780 (454 en F1, 449 en F2, 877 en F3)                      |
| Encoder                | intfloat/multilingual-e5-small (384 dim, 117,65M parámetros) |
| Índice vectorial       | FAISS IndexFlatIP, ~508 MB, búsqueda exacta                  |
| Agregación a documento | max pooling (Sección 8.6)                                    |

## 2. Preprocesamiento y estrategia de chunking

La extracción de texto se implementó por separado para cada fenómeno (F1/F2/F3), con manejo específico por formato: PDF vía extracción de texto preservando orden de lectura, CSV/XLSX fila por fila conservando el nombre de columna como contexto (evitando el colapso de espacios en blanco que destruye los límites de fila y dispara una explosión de chunks en exportaciones bibliográficas), PBF/MVT vía mapbox-vector-tile conservando una sola versión por elemento para no duplicar entre niveles de zoom, e imágenes vía OCR con pytesseract (lang='spa+eng'). Cada documento recibió detección de idioma (langdetect); los falsos positivos conocidos del detector sobre texto corto o estructurado (español marcado como catalán, alemán o rumano en fuentes CENIA y datos PBF) se retuvieron explícitamente en vez de excluirse.

El chunking combina una estrategia consciente del formato —filas de CSV/XLSX/PBF divididas por salto de línea, prosa dividida por oración con NLTK— con un tope duro de 512 tokens (la ventana máxima de multilingual-e5-small) aplicado mediante repaquetado por tokens que siempre retrocede al último punto de corte oracional completo, cumpliendo el requisito de completitud lingüística de la Sección 3.3. El resultado final: 330.416 chunks, 0 excepciones, con un máximo observado de 250 palabras por chunk en todo el corpus (promedio 118,2) —lo cual, como se detalla en la Sección 5, hace que la regla de división por límite de 250 palabras de la Sección 9.2.1 nunca se active en la práctica sobre este corpus, aunque el módulo de recuperación la implementa por completitud.

## 3. Selección del encoder

Se seleccionó intfloat/multilingual-e5-small (384 dimensiones, ventana de 512 tokens, licencia MIT) por soporte multilingüe nativo de español/inglés/portugués —requisito explícito de la Sección 4.3, dado que el corpus mezcla los tres idiomas— y por eficiencia computacional frente a alternativas más grandes, dado el volumen del corpus. La familia se comparó contra multilingual-e5-base, paraphrase-multilingual-mpnet-base-v2 y BAAI/bge-m3 mediante un script de benchmarking propio (NDCG@10 y F1@3 contra un ground truth interno). Un detalle de implementación crítico de la familia E5: exige los prefijos "passage: " y "query: " en el momento de codificar —documento y consulta, respectivamente— para que ambos habiten el mismo espacio semántico. Estos prefijos se aplican solo al codificar y nunca se persisten en el campo texto de la metadata, que permanece "sin modificaciones" como exige la Tabla 1 del spec; el presupuesto de tokens de la etapa de chunking ya reservó espacio

para el prefijo, por lo que ningún chunk quedó sin margen al agregárselo.

## 4. Índice vectorial FAISS

Se construyó un único índice `IndexFlatIP` sobre vectores normalizados a norma unitaria (habilitando búsqueda exacta por similitud coseno vía producto interno, Sección 8.2), siguiendo la recomendación explícita del spec para el volumen de este reto (Sección 5.2). El archivo resultante pesa ~508 MB para 330.416 vectores de 384 dimensiones. La codificación completa del corpus tomó 9,5 minutos sobre GPU (580 chunks/segundo). El orden de inserción en el índice se mantuvo idéntico al orden de lectura de `metadata.jsonl`, garantizando que el id interno de FAISS coincida con el número de línea del almacén de metadata (Sección 1.4).

### 4.1. Validación del índice

- Estructural: tipo de índice, dimensión (384) y conteo de vectores verificados contra las líneas de metadata.
- Normalización: muestra de 200 vectores con norma unitaria confirmada (mín/máx = 1,000000).
- Identidad vector↔texto: 20 posiciones aleatorias reconstruidas desde el índice y recalculadas desde cero a partir del texto de esa misma línea —similitud mínima obtenida: 1,000000, confirmando que no hay desalineación de contenido, no solo de conteo.
- Recuperación multilingüe: prueba de humo con una consulta por fenómeno en tres idiomas distintos (es/en/pt) confirmó recuperación cruzada de idioma —por ejemplo, una consulta en portugués sobre dinámicas territoriales recuperó correctamente documentos en español.

## 5. Módulo de recuperación (generador.py)

Implementa el flujo de la Sección 8.1 sin usar en ningún punto un modelo generativo (Sección 8.3): la consulta se codifica con el mismo encoder e idéntico prefijo E5, se normaliza, y se busca en FAISS. El tamaño del pool de candidatos  $k$  es dinámico: arranca en 30 y se duplica hasta 200 si hace falta, garantizando que toda consulta obtenga exactamente 3 documentos y 10 fragmentos —requisito estricto de la Sección 9.3.2, donde un conteo incorrecto se penaliza o descarta.

### 5.1. Agregación a documento (Sección 8.6)

Se adoptó **max pooling** —el score del fragmento más relevante representa al documento— tras descartar dos alternativas propias mediante validación empírica (detalle en Sección 7).

### 5.2. Límite de 250 palabras por fragmento (Sección 9.2.1)

Los chunks recuperados que superan 250 palabras se dividen en sub-fragmentos mediante tokenización de oraciones (NLTK) empaquetadas de forma voraz hasta el límite, preservando siempre límites oracionales completos; todas las sub-piezas de un mismo chunk comparten el mismo `chunk_id` y ocupan posiciones de ranking independientes, como exige el spec. En la práctica esta ruta no se activa sobre el corpus actual (ningún chunk supera 250 palabras), pero se implementó y validó con texto sintético para no dejar un caso límite sin cubrir.

### 5.3. Autovalidación de salida

`generador.py` valida su propia salida al terminar: cuenta líneas de `resultados.jsonl`, exige exactamente 3 documentos y 10 fragmentos por consulta, y verifica que ningún fragmento exceda el límite de palabras —una red de seguridad barata contra errores de esquema antes de la entrega.

## 6. Experimentos de agregación: hipótesis probadas y descartadas

Durante la validación se detectó un sesgo sistemático: documentos con muchísimos chunks (p. ej. el Índice Latinoamericano de IA, hasta 975 chunks frente a una mediana de 7 en todo el corpus) dominaban el top-3 de consultas específicas incluso cuando existía un documento más pequeño y preciso con el contenido correcto —confirmado con la consulta "¿Qué desafíos plantea el uso de inteligencia artificial en las operaciones espaciales?", donde el sistema devolvió 0/3 documentos del fenómeno correcto pese a que existían 86 chunks de contenido de IA aplicada al dominio espacial en el corpus. La causa raíz identificada: con *max pooling*, el score de un documento es el máximo entre sus chunks candidatos —el mejor resultado de  $N$  intentos—, y un documento con más chunks tiene estadísticamente más oportunidades de que uno de ellos puntúe alto por azar (estadística de valores extremos), sin que eso implique mayor relevancia real.

| Hipótesis                                         | Mecanismo                                          | Resultado empírico (50 consultas)                                                                                                                                 | Decisión   |
|---------------------------------------------------|----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Penalización $\log(n\_chunks)$                    | $score - \beta \cdot \log(n\_chunks)$              | Fenómeno 1: 85%→48% (documentos con tamaño típico de F1 penalizados injustamente frente a F3, cuya mediana de chunks/doc es 2 vs. 14 en F1)                       | Descartada |
| + umbral (solo penaliza exceso sobre 100 chunks)  | $score - \beta \cdot \max(0, \log(n) - \log(100))$ | Sin cambio real: F1 se mantuvo en 48%. Total agregado 89,3%→78,7% (-10,6 pp)                                                                                      | Descartada |
| Media de los mejores k chunks por doc (topk_mean) | promedio de los 3 chunks más altos por doc         | Empeoró el caso diagnóstico: premió amplitud genérica (doc grande con muchos chunks decentes) sobre enfoque específico (doc chico con pocos chunks pero certeros) | Descartada |

Las tres variantes se probaron contra las 50 consultas reales (no contra casos aislados) usando un proxy de coincidencia de fenómeno documento↔consulta, validado además con casos de depuración individuales antes de la corrida completa. Las tres empeoraron el resultado agregado frente al *max pooling* sin modificar, así que se revirtieron. La configuración final entregada usa *max pooling* puro (equivalente a  $\beta=0$ ). El código de las variantes probadas se conservó por separado (generador\_v2.py) para trazabilidad, pero no forma parte de la ruta usada para generar la entrega final.

## 7. Validación contra ground truth y limitaciones conocidas

El archivo FASE\_ORDENADA\_CODEFEST.xlsx provisto por el equipo organizador contiene relevancia verificada (pregunta, fragmento y documento fuente) para 7 de las 50 consultas de evaluación (q002, q003, q004, q033–q036), con un esquema de identificadores propio que se cruzó contra el doc\_id interno del equipo vía coincidencia normalizada de nombre de archivo (campo fuente). Sobre esta muestra: **1/7 consultas (14%)** obtuvieron el documento correcto en el top-3, y **3/7 (43%)** lo obtuvieron en el top-3 o entre los 10 fragmentos. Es una muestra pequeña —no reemplaza el NDCG@10/F1@3 oficial sobre las 50 consultas, cuyo ground truth no es público durante el reto— pero es evidencia real, no un proxy.

El patrón de falla identificado en la Sección 6 se confirmó en esta muestra independiente: las consultas que combinan el término genérico "inteligencia artificial" con un calificador técnico específico ("sistemas no tripulados", "conflictos recientes", "operaciones espaciales") tienden a recuperar el Índice Latinoamericano de IA —un reporte extenso y genérico— en lugar de la fuente específica correcta (informes DAIO, CSIS), incluso cuando esta última existe en el corpus con contenido directamente relevante. Se identifica como **limitación conocida** del enfoque de recuperación puramente vectorial con un encoder compacto (117M parámetros): el término genérico domina la señal semántica sobre el calificador específico.

### Trabajo futuro propuesto

La dirección identificada con mayor evidencia de impacto es recuperación híbrida (búsqueda léxica tipo BM25 combinada con la búsqueda vectorial actual vía Reciprocal Rank Fusion, Sección 8.4): un match léxico exacto de términos técnicos específicos no depende del tamaño del documento ni de matices que un encoder compacto no capture. No se implementó en esta entrega por restricción de tiempo; queda documentado como el próximo paso concreto.